MyScoutee
QUALITY ASSURANCE DOCUMENTATION

LOCAL PAIR QA AND IMMEDIATE REPAIR

MyScoutee Local VM QA Test Guide

Codex Pro 20 operates the technical QA workflow and implements requested corrections in the same pair session. The human developer or reviewer checks real user flow and UX, catches typical bugs, and requests fixes without having to learn or execute the entire runbook.

Indicative cadence, not a delivery guarantee: week 1 targets the 30–35 hardest multi-model cases; week 2 completes the remaining core campaign (about two weeks total for core); week 3 covers external/optional QA; week 4 covers locally executable integrations that still require a provisioned external provider, merchant account, or domain.

DOCUMENT ID	DOCUMENT VERSION
MSC-QA-001	1.0.0
APPLICABLE SOFTWARE VERSION	**STATUS**
MyScoutee 1.0.0 QA campaign	Working pair-QA snapshot
SNAPSHOT DATE	**LANGUAGE**
2026-08-19	EN

1. Start Here

More than delayed QA: pair review plus immediate repair

Codex Pro 20 operates the replayable checkpoints, opens the managed browsers, and runs the oracles and evidence workflow. The human developer/reviewer inspects flow and UX, then accepts the result or immediately asks Codex to fix it. Codex uses the existing dev pipeline, replays the checkpoint, and reopens the browser. This same-session QA–repair–recheck loop avoids a delayed cross-team defect queue.

Why human review remains essential: rapid development can outpace expectations, AI QA can hallucinate expected behavior, and E2E proves only encoded assertions. Human review grounds the result in the real product experience and catches typical bugs the automation did not model.

Read the live pointer before touching state

Codex starts from `CURRENT_QA_SESSION.md` and its linked checkpoint. The human waits for the managed browser and never prepares or repairs fixture state. The ledger owns status, the QA pack owns evidence, and the live pointer overrides this PDF.

Current campaign snapshot

59 / 127
Core passed · 68 remaining

3 / 97
External/optional passed · 94 remaining

62 / 224
Total passed · 162 remaining

Active checkpoint	`CHAT-LIST-001` — chat list and contexts
Saved boundary	`phase=fixture-ready`; manual execution paused. Fixture readiness is not a PASS.
Prepared actor/state	Casey Bridge; four controlled rows: Event, Group, Service and App Support; unread one per row; All=4 and Sub-event=0.
Already proven	Authenticated Java ASC/DESC order, exact context totals, stable headers, and a byte-identical post-read Mongo snapshot.
Paired human overview still due	Inspect All and each populated context, sort once, close/reopen, and confirm the same order and counters. Do not open a thread.
After closure	Return to the reopened Tournament block at `TOURNAMENT-MEMBER-001`. Tournament is currently 9/14.

2. Prepare the Ubuntu LTS VM

Download only from the official vendors

- Download an **amd64 Ubuntu LTS ISO** from ubuntu.com/download. Ubuntu Desktop LTS is the simplest all-in-one manual-QA workstation because it can display the managed Chrome windows. Ubuntu Server LTS is suitable when Chrome/CDP remains on the trusted host and reaches the VM through SSH forwarding.
- Install VMware Workstation Pro, currently offered free by VMware/Broadcom, or the VirtualBox base package. Confirm the current vendor licence and organizational policy before use.
- VirtualBox's base package and Extension Pack have different licences. Basic QA does not require the Extension Pack. Android USB 2/3 pass-through may require it; the host-side ADB plus SSH-forward route below avoids that dependency.
- Record the ISO filename and published SHA-256. Do not use an ARM image. “Latest LTS” is not automatically the release-qualified host; retain the exact LTS in the QA evidence.

VM baseline

Resource	Minimum local smoke	Recommended full dev/manual QA
CPU	4 amd64 vCPU	8 amd64 vCPU
Memory	6 GiB	16 GiB with host headroom
Disk	40 GiB dynamically allocated	80 GiB dynamically allocated
Network	NAT	NAT plus a documented host-only/admin path if needed

Enable hardware virtualization in firmware. Create an Ubuntu 64-bit VM, attach the ISO, use a named non-root administrative account, select OpenSSH during installation, and take a clean post-install snapshot before MyScoutee data exists. Do not clone a VM after MyScoutee deployment identity or provider credentials have been created.

Guest preparation

```
sudo apt update
sudo apt upgrade
sudo apt install openssh-server git curl ca-certificates jq adb
sudo sshd -t
sudo systemctl enable --now ssh
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Install Docker Engine and Docker Compose v2 from an approved Ubuntu/Docker package source. A packaged MyScoutee install resolves these dependencies through APT; a source dev stack needs them before its first Compose build.

3. SSH, Port Forwarding, and Network Boundary

Prefer a private NAT/host-only route. Restrict guest TCP 22 to the physical host or approved administration network. If the hypervisor NAT cannot route directly to the guest, create one narrow rule such as host `127.0.0.1:2222` to guest TCP `22`.

1. Read the guest SSH host-key fingerprint from the VM console.
2. Compare it on the physical host before accepting the first connection.
3. Use a dedicated key and a named non-root account with only the required `sudo` authority.
4. Never expose SSH to `0.0.0.0/0`; never expose MongoDB, Redis, MinIO, Java, or registry container ports.

```
ssh -i /absolute/path/to/id_ed25519 \
  -p 2222 qa-admin@127.0.0.1
```

For the repository client tools, copy `packaging/client-tools/install.env.example` to a gitignored `install.env`, set mode 0600, and use strict host-key checking after the fingerprint has been pinned:

```
REMOTE_HOST=127.0.0.1
REMOTE_USER=qa-admin
REMOTE_PORT=2222
REMOTE_KEY_FILE=/absolute/path/to/id_ed25519
REMOTE_HOST_KEY_CHECKING=yes
```

Optional loopback-only application tunnel

```
LOCAL_FORWARD_PORTS_OVERRIDE=18080:80,18443:443 \
  ./packaging/client-tools/forward-localhost.sh
```

This creates host-loopback forwards to VM-loopback HTTP/HTTPS. The helper refuses non-loopback binds. For a managed QA browser that is hard-wired to `http://localhost`, either run it inside an Ubuntu Desktop VM or use the default local 80/443 mapping with `sudo -E ./forward-localhost.sh`. A tunnel is useful for isolation but does not prove public DNS, TLS, redirect, WebSocket, NAT, or Internet reachability.

4. Choose One Runtime Path

A. Current manual-QA campaign

Use the source development stack with hot deploy. Place the authorized `myscoutee-registry` checkout beside the backend checkout and initialize the frontend submodule.

```
git clone --recurse-submodules \
  https://github.com/fssrepository/myscoutee-backend.git
# Place the authorized registry checkout at ../myscoutee-
registry
cp server/.env.secrets.example server/.env.secrets
chmod 0600 server/.env.secrets
# Replace every placeholder in server/.env.secrets
cd server
docker compose up --build
```

The Compose build is a first-start action only. Once the dev stack and watchers exist, do not launch another Compose stack, Gradle/npm build, test runner, MongoDB, server, watcher, or Chrome profile set beside them.

B. Package/deployment smoke

Use an exact approved Debian package and matching client bundle. Authenticate the release signature, pinned full key fingerprint, size, SHA-256, package identity, and version before upload.

```
./install.sh \
  /approved/path/myscoutee_1.0.0_amd64.deb

node verify-deployment/run.mjs \
  --url https://deployment.example \
  --expected-version 1.0.0
```

The installer uploads through SSH, installs with APT, and invokes package-owned host qualification. The external verifier is a separate read-only client command. It never disables TLS verification. Package smoke does not replace the current source/manual-QA campaign.

> ### Do not mix the paths
>
> A development Compose stack, source-production stack, and APT/systemd-installed package have different state, build, TLS, and lifecycle boundaries. Record which path is under test. Never interpret a source test as release qualification.

Running-stack preflight

```
cd server
docker compose ps
free -h
docker stats --no-stream
docker logs --since 5m myscoutee
docker logs --since 5m frontend-sync
docker logs --since 5m nginx
```

If a watcher owns compilation, trigger only that existing owner and inspect its log. If it is stuck or owns a cache lock, identify and restart only that exact owner. Do not create another cache or temporary build path.

5. Pair-QA Operating Contract and Resume

One testcase, a human click guide, and replayable substages

The CSV/XLSX ledger selects the testcase and execution order. Each selected testcase has one stable English click guide under `guides/qa/checklists/`. That guide tells the paired human where to click, what to enter, what to observe, and which phrase marks a machine-oracle boundary. It never stores live status, checked boxes, transient fixture IDs, or Codex evidence.

Functional authenticity is mandatory

Pair QA tests the real assembled product behavior, not a fixture-authored imitation of the expected result. A fixture may prepare production-reachable source data and ordinary persisted development configuration. It must never fabricate or backdate runtime state, `lastRunAtIso`, Runner history, counters, Metrics or audit events, completion state, notifications, projections, or a user-visible outcome. Any attempt that does so is **NOT EXECUTED / invalid fixture** and must be reset and redesigned before evidence is accepted.

For scheduler QA, Codex preserves the genuine last-run state, changes the real persisted target frequency to the documented short development interval—normally one minute—and waits for the already-running scheduler and production worker. Direct worker calls, artificial due-state timestamp writes, fabricated history or telemetry, and the manual `Run` action are forbidden as evidence.

1. Codex prepares guarded `stage`, read-only `status`, and exact `reset` operations; proves the selected baseline; and opens the isolated actor browsers.
2. Fixture preparation does not replace a product transaction named by the checklist. If creation is under test, the human creates the domain record through the UI and Codex captures its generated identity afterward.
3. A complex testcase may have meaningful named substages such as `pre-create`, `post-create`, `slot-override`, or `resource-assigned`. They remain inside one testcase ID and are not created around every click.
4. The paired human performs and judges every visual/interaction step. At each named stop, Codex checks the independent API/backend/persisted oracle and records evidence outside the checklist.
5. After a discrepancy, freeze the boundary, repair the smallest owning scope, restore the nearest accepted substage, and replay the shortest causal tail. Run the complete journey when the human explicitly requests it or the failure contaminated an earlier boundary.

The live handoff always identifies the testcase, checklist section, substage, fixture phase, actor/browser state, exact next human action, stop phrase, and pending machine oracle. A new session resumes through one read-only status gate instead of reconstructing the campaign from conversation history.

Checkpoint-specific example: resume `CHAT-LIST-001`

The commands below define the reproducible operator path that **Codex Pro 20 executes automatically** from the backend repository root. They are included for auditability and exceptional takeover, not as a command list the paired human must learn. The Docker wrapper must reach the already-running `mongodb` container; it never starts or restarts the stack.

Read-only continuity gate

```
bash server/docker/scripts/manual-qa-checkpoints/manual-qa-checkpoints-docker.sh \
  status chat-domain CHAT-LIST-001
```

Result	Pair-QA continuation
Matching `phase=fixture-ready` and every validation true	Codex continues without staging again and reuses a live managed browser only when its manifest and process are still valid.
`phase=not-staged`	Codex replays the checkpoint by staging once, validating status again, and launching the managed checkpoint browser.
Partial, foreign, or unexpected state	Codex stops and records the exact mismatch; neither partner repairs state through ad-hoc Mongo writes or the UI.
Docker socket denied / “MongoDB is not running” ambiguity	Codex fixes Docker access and reruns the same status command without starting a second MongoDB or Compose stack.

Codex replay route — fresh/stale runtime only

```
bash server/docker/scripts/manual-qa-checkpoints/manual-qa-checkpoints-docker.sh \
  stage chat-domain CHAT-LIST-001

bash server/docker/scripts/manual-qa-checkpoints/manual-qa-checkpoints-docker.sh \
  status chat-domain CHAT-LIST-001

guides/qa/tools/open-checkpoint-browser-sessions.sh CHAT-LIST-001
```

Codex does not run `stage` when the first status already proves the saved boundary. When replay is required, Codex invokes the launcher, which opens the isolated Casey Chrome window and writes `/tmp/myscoutee-qa-browser-sessions/CHAT-LIST-001.json`. The paired human does not launch or reconnect the browser manually.

CDP preflight and read oracle

```
node guides/qa/tools/verify-chat-list-read-oracle.mjs \
  --manifest /tmp/myscoutee-qa-browser-sessions/CHAT-LIST-001.json \
  --actor-user-id 650000000000000000000061
```

```
bash server/docker/scripts/manual-qa-checkpoints/manual-qa-checkpoints-docker.sh \
  status chat-domain CHAT-LIST-001
```

Human flow/UX and typical-bug overview — the only manual step here

- [ ] Wait until Codex opens and hands over the managed Casey window; in that window, open **Activities → Chats**.
- [ ] Confirm All contains exactly four controlled rows and unread is one per row.
- [ ] Inspect Event, Group, Service and App Support: one matching row in each. Confirm Sub-event is zero.
- [ ] Sort once and confirm the exact date order remains coherent.
- [ ] Close and reopen Chats; confirm the same All order, headers and counters.
- [ ] **Do not open a chat thread.** Read acknowledgement belongs to `CHAT-READ-001`.
- [ ] If anything differs or looks wrong, the human developer/reviewer asks Codex to fix it immediately. Do not compensate by repairing data or runtime state manually; review the replayed checkpoint again.

6. CDP and Android USB Debugging

Chrome DevTools Protocol (CDP)

The managed launcher assigns a loopback-only ephemeral DevTools port and records it in the checkpoint manifest. In default manual QA, the human controls navigation and checklist actions; after a reported defect, Codex may perform targeted read-only diagnosis.

When the user explicitly delegates a specified full flow or checkpoint, Codex may operate those managed sessions through CDP, perform the checklist, repair defects and replay the affected steps. That authorization remains active for the delegated scope until completion or revocation. Fixture baselines, the versioned launcher, resource limits and independent persistence oracles still apply. Record automated observations as such; never invent a human verdict or manufacture passing runtime state.

Event-format and partner API follow-up

`EVENT-MODE-REGRESSION-001` covers Standard/Tournament isolation, Speed meeting runtime, Explore preferences, external event links and local persistence. Its stable guide is `guides/qa/checklists/event/EVENT-MODE-REGRESSION-001.md`. The extension in `guides/qa/checklists/integration/INTEGRATION-API-001.md` covers participant invitation claims and completed encounter reads/webhooks. The earlier API PASS does not establish the new extension; record a separate dated result.

```
QA_CDP_PORT=$(jq -r '.sessions[0].devToolsPort' \
  /tmp/myscoutee-qa-browser-sessions/CHAT-LIST-001.json)

node guides/qa/tools/chrome-target-network.mjs inspect \
```

```
  --user-id 650000000000000000000061 \
  --port "$QA_CDP_PORT"
```

- Close only the temporary inspector WebSocket. Never call `browser.close()` or a browser-level close command against a managed QA Chrome.
- Do not expose the CDP port beyond loopback.
- Offline `--hold` and geolocation overrides are permitted only when the active checkpoint explicitly requires them; always restore online state in cleanup.
- The local dev launcher may tolerate the local bootstrap certificate. The production deployment verifier never permits insecure TLS.

Android USB debugging and ADB reverse

This is required only for a checkpoint that names a physical Android device, such as QR scanning, cold-start push routing, camera, or WebView behavior. It is not part of `CHAT-LIST-001`.

1. Enable Android Developer options and USB debugging.
2. Connect the phone and accept the exact host/VM RSA authorization prompt.
3. Attach the USB device to the Ubuntu VM in VMware/VirtualBox, or keep ADB on the physical host and combine it with the SSH tunnel.

ADB runs inside the QA VM

```
adb devices -l
adb reverse tcp:8080 tcp:80
adb reverse --list
# On the phone: http://localhost:8080
```

ADB stays on the physical host

```
# First keep the SSH forward host 127.0.0.1:18080 -> VM 127.0.0.1:80 active.
adb devices -l
adb reverse tcp:8080 tcp:18080
adb reverse --list
# On the phone: http://localhost:8080
```

The device must show state `device`, not `unauthorized`. Reapply the reverse after an ADB daemon restart, USB reconnect, phone restart, or VM USB reattachment. Remove only the exact mapping during cleanup with `adb reverse --remove tcp:8080`.

7. Evidence and Pair-QA Handover Closeout

Close the active checkpoint only after all evidence agrees

- [ ] The paired human accepts the visible result or requests an immediate correction.

- [ ] For a correction, Codex uses the existing dev pipeline, replays the checkpoint, reopens the managed browser, and returns it for another human review.
- [ ] After human acceptance, Codex runs the required Docker/API/persistence oracles and requires exact IDs/order/totals plus the documented write/no-write boundary. Targeted CDP is used only when diagnosis or a checkpoint-specific control requires it. UI alone is not PASS.
- [ ] Codex writes the QA-pack evidence, updates CSV/XLSX and README totals, and moves the live pointer in `CURRENT_QA_SESSION.md`.
- [ ] Only then does Codex reset the fixture and advance to `TOURNAMENT-MEMBER-001`.

Minimum pair-QA handover

Codex Pro 20 carries forward the testcase/checkpoint ID, checklist path and section, named substage, fixture phase, actor/browser state, environment/profile/database, last oracle, exact next human action, stop phrase, pending machine oracle, and running processes. The human only needs the observable task, its PASS conditions, and the ability to request a correction; a new human is not expected to reconstruct the automation workflow.